

Relatório Técnico — Stella (STL)

Status: Documento oficial

Escopo: Descrição técnica da implementação atual do protocolo, do modelo econômico e da API da Stella.

1. Visão Geral

A Stella (STL) é uma rede blockchain baseada no modelo de **UTXO** (*Unspent Transaction Output*), com emissão monetária controlada por um mecanismo de mineração por Proof of Work (PoW). O sistema é composto por um servidor de aplicação (API HTTP em Flask), um módulo de núcleo do protocolo (estrutura de blocos, transações, mineração e validação), um módulo de persistência (PostgreSQL) e um módulo de comunicação entre nós.

A unidade monetária principal é o **STL**, subdividida em uma unidade mínima chamada **Saint**, onde 1 STL equivale a 100.000.000 Saints.

2. Arquitetura do Sistema

O sistema é organizado nos seguintes componentes principais:

Interface Web / API (Flask)



Camada de Aplicação (rotas, sessão de usuário)



Núcleo do Protocolo (Blockchain, Blocos, Transações, UTXOs, Mineração, Criptografia)



Persistência (PostgreSQL: blocos, UTXOs, carteiras, mempool, usuários)



Comunicação entre Nós (registro de nós, broadcast de transações e blocos via HTTP)

A classe central Blockchain mantém em memória o estado corrente da cadeia (chain), o conjunto de transações pendentes (current_transactions), o conjunto de UTXOs disponíveis (available_utxos) e a lista de nós conhecidos (nodes). Esse estado é inicializado a partir do banco de dados na criação do objeto e atualizado conforme blocos e transações são processados.

3. Tecnologias e Dependências

Categoria	Tecnologia
Linguagem	Python
Framework web	Flask
Banco de dados	PostgreSQL, via biblioteca psycopg2
Criptografia de curva elíptica	biblioteca cryptography (curva SECP256K1, assinatura ECDSA com SHA-256)

Categoria	Tecnologia
Hashing	hashlib (SHA-256)
Hash de senha	hashlib.scrypt (com sal aleatório)
Criptografia simétrica	cryptography.fernet.Fernet, para proteger chaves privadas armazenadas
Comunicação entre nós	biblioteca requests (HTTP)
Configuração de ambiente	python-dotenv
Identificadores	uuid4 (identificação de usuários e do nó local)

4. Estrutura do Projeto

O código está organizado em módulos com responsabilidades bem definidas:

- ``api/app.py`` — aplicação Flask e definição das rotas HTTP.
- ``core/block.py`` — estrutura de blocos, hashing e validação estrutural.
- ``core/blockchain.py`` — orquestração da cadeia, mineração, validação de cadeia e resolução de conflitos.
- ``core/transaction.py`` — criação, assinatura e validação de transações.
- ``core/utxo.py`` — criação e verificação de UTXOs, cálculo de saldo.
- ``core/wallet.py`` — geração de carteiras, endereços e assinatura de transações do usuário.
- ``core/crypto.py`` — funções primitivas de geração de chaves, assinatura e verificação.
- ``core/mining.py`` — regras de dificuldade, Proof of Work, recompensa e emissão.
- ``core/auth.py`` — hashing e verificação de senhas de usuário.
- ``core/user.py`` — representação de um usuário do sistema.
- ``network/node.py`` — identificação do nó local e transmissão de transações/blocos.
- ``network/consensus.py`` — registro de nós remotos e resolução de conflitos entre cadeias.
- ``persistence/database.py`` — acesso ao PostgreSQL (blocos, UTXOs, carteiras, mempool, usuários).
- ``persistence/user_repository.py`` — criação e consulta de usuários e suas carteiras.
- ``config.py`` — parâmetros centrais do protocolo e da aplicação.
- ``main.py`` — ponto de entrada da aplicação.

5. Carteiras e Modelo de Custódia

Uma carteira (Wallet) é composta por uma chave privada de curva elíptica SECP256K1, gerada com `ec.generate_private_key`, e sua respectiva chave pública. O **endereço** da carteira é obtido

aplicando SHA-256 sobre a chave pública codificada no formato comprimido (X9.62), resultando em uma string hexadecimal.

A criação de uma carteira ocorre automaticamente no momento do registro de um usuário (`create_user`). A chave privada é serializada em formato PEM, criptografada simetricamente com Fernet e armazenada na tabela `wallets` do PostgreSQL. A chave simétrica usada nessa criptografia é gerada automaticamente na primeira execução e mantida em um arquivo local (`database/wallet.key`).

Esse desenho caracteriza um **modelo custodial**: as chaves privadas dos usuários são geradas, armazenadas e utilizadas pelo próprio servidor da aplicação para assinar transações em nome do usuário autenticado, e não diretamente pelo usuário em um ambiente externo.

6. Blockchain

6.1 Estrutura de um Bloco

Cada bloco é um dicionário com os seguintes campos, definidos em `new_block`:

Campo	Descrição
<code>index</code>	posição do bloco na cadeia
<code>timestamp</code>	momento de criação do bloco
<code>transactions</code>	lista de transações incluídas no bloco
<code>proof</code>	valor de prova de trabalho encontrado para o bloco
<code>previous_hash</code>	hash do bloco anterior

6.2 Bloco Gênesis

O primeiro bloco da cadeia é criado automaticamente quando não existem blocos persistidos, com `index=1`, lista de transações vazia, `proof=100` e `previous_hash=1`.

6.3 Hashing e Encadeamento

O hash de um bloco é calculado por `hash(block)`, que serializa o bloco em JSON com chaves ordenadas (`json.dumps(block, sort_keys=True)`) e aplica SHA-256 sobre o resultado. Cada bloco referencia o hash do bloco anterior por meio do campo `previous_hash`, formando a cadeia. A validação de encadeamento verifica se `bloco.previous_hash` é igual ao hash calculado do bloco anterior.

6.4 Validação Estrutural

`valid_block` verifica se um bloco contém todos os campos obrigatórios e se seus tipos são consistentes (`index` inteiro, `timestamp` numérico, `transactions` lista, `proof` inteiro, `previous_hash` inteiro ou string).

6.5 Validação de Cadeia

A validação de uma cadeia completa (`valid_chain`) percorre os blocos em sequência, verificando, para cada bloco: estrutura válida, encadeamento correto com o bloco anterior, validade da prova de trabalho conforme a dificuldade recalculada a partir da porção da cadeia existente até aquele bloco, presença de exatamente uma transação coinbase válida como primeira transação do bloco, e validade de todas as demais transações contra o conjunto de UTXOs disponíveis reconstruído a partir da própria cadeia sendo validada.

6.6 Validação de um Bloco Recebido

Quando um bloco é recebido de outro nó (`valid_received_block`), a validação é feita contra o estado atual da cadeia local: encadeamento com o último bloco local, prova de trabalho conforme a dificuldade atual, validade das transações e da transação coinbase.

7. Transações

7.1 Modelo UTXO

A Stella utiliza o modelo de UTXO: o saldo de um endereço é a soma dos valores de todos os UTXOs (saídas não gastas) que o referenciam como proprietário (`owner`). Uma transação consome um ou mais UTXOs existentes como entradas e cria novos UTXOs como saídas.

7.2 Estrutura de uma Transação

Uma transação contém:

- `inputs`: lista de UTXOs consumidos;
- `outputs`: lista de saídas, cada uma com `owner` (endereço de destino) e `amount`;
- `fee`: taxa protocolar aplicável (ver seção 10);
- `transaction_id`: identificador calculado por SHA-256 sobre os dados da transação;
- `public_key`: chave pública do remetente, codificada em PEM e hexadecimal;
- `signature`: assinatura ECDSA sobre os dados da transação.

Transações do tipo coinbase (recompensa de mineração) não possuem entradas, `public_key` nem `signature`, e incluem um campo adicional `block_index` indicando o bloco ao qual pertencem.

7.3 Criação

A criação de uma transação (`new_transaction`) segue as seguintes etapas:

1. Verifica que cada entrada corresponde a um UTXO efetivamente disponível.
2. Verifica que todos os valores de saída são múltiplos válidos da unidade mínima (Saint).
3. Calcula a taxa protocolar aplicável, quando um índice de bloco é informado.
4. Verifica que a soma das saídas mais a taxa não excede a soma das entradas.
5. Caso haja diferença entre entradas e saídas mais a taxa, gera uma saída de troco destinada ao proprietário da primeira entrada.

6. Calcula o `transaction_id` por SHA-256 sobre os dados da transação.
7. Remove os UTXOs de entrada do conjunto disponível e os marca como consumidos.
8. Cria novos UTXOs para cada saída e os adiciona ao conjunto de UTXOs disponíveis.

Uma característica relevante desse processo é que os UTXOs de saída de uma transação recém-criada localmente já passam a integrar o conjunto de UTXOs disponíveis imediatamente, antes de a transação ser incluída em um bloco. Isso permite que saídas de transações ainda não confirmadas sejam utilizadas como entrada de novas transações.

Na carteira (`Wallet.create_transaction`), a seleção de entradas utiliza a totalidade dos UTXOs pertencentes ao endereço do remetente; o valor excedente ao pagamento e à taxa retorna como troco para o próprio remetente.

7.4 Assinatura

`sign_transaction` anexa a chave pública do remetente (em PEM, codificada em hexadecimal) à transação e assina, com a chave privada correspondente, a representação textual dos demais campos da transação (ECDSA com SHA-256 sobre a curva SECP256K1). A assinatura resultante é armazenada em hexadecimal no campo `signature`.

7.5 Validação

`valid_transaction` executa as seguintes verificações:

1. Presença de entradas, saídas, assinatura e chave pública.
2. A chave pública informada é uma chave EC válida.
3. O endereço derivado da chave pública (SHA-256 da chave comprimida) corresponde ao proprietário de todas as entradas — isso comprova que quem assina a transação é o titular dos UTXOs que está gastando.
4. Os valores de saída e a taxa respeitam a unidade mínima (Saint).
5. Cada entrada corresponde a um UTXO efetivamente disponível.
6. Quando um índice de bloco é informado, a taxa declarada corresponde exatamente à taxa calculada pelo protocolo para aquele contexto.
7. A soma das saídas mais a taxa não excede a soma das entradas.
8. A assinatura é criptograficamente válida sobre os dados da transação.

8. Proof of Work

8.1 Prova de Trabalho

A validade de uma prova (`valid_proof`) é verificada concatenando a prova do bloco anterior com a prova candidata, aplicando SHA-256 e checando se o hash resultante inicia com uma quantidade de zeros igual à dificuldade vigente. A busca pela prova (`Blockchain.proof_of_work`) incrementa um contador a partir de zero até encontrar um valor que satisfaça essa condição — um processo de força bruta (Hashcash).

8.2 Ajuste de Dificuldade

A dificuldade (get_difficulty) é recalculada a partir do intervalo de tempo entre os dois blocos mais recentes da cadeia: $\text{dificuldade} = \text{dificuldade_base} \times (\text{tempo_alvo_do_bloco} / \text{tempo_observado_entre_os_dois_últimos_blocos})$, arredondada para o inteiro mais próximo, com valor mínimo de 1. Caso a cadeia tenha menos de dois blocos, ou o intervalo observado seja inválido, é utilizada a dificuldade base definida em configuração.

9. Mineração

Minerar um bloco (Blockchain.mine_block) segue as etapas:

1. Calcula a dificuldade vigente a partir da cadeia atual.
2. Executa a busca por prova de trabalho a partir da prova do último bloco.
3. Calcula a recompensa de bloco correspondente ao índice do novo bloco e soma o total de taxas das transações pendentes na mempool.
4. Cria a transação coinbase, destinando a soma da recompensa e das taxas ao endereço do minerador.
5. Valida a transação coinbase conforme as regras de emissão.
6. Inclui a transação coinbase como primeira transação do novo bloco, junto às demais transações pendentes.
7. Adiciona o bloco à cadeia, persiste-o, processa as transações confirmadas (removendo UTXOs gastos e registrando novos UTXOs), libera a mempool das transações incluídas, processa recompensas de mineração já maturadas e transmite o bloco aos nós conhecidos.

A mineração é executada de forma síncrona, dentro da própria requisição HTTP feita à rota correspondente, por um usuário autenticado.

10. Emissão Monetária e Regras Econômicas

10.1 Parâmetros Implementados

Parâmetro	Valor
Supply máximo	20.000.000 STL
Recompensa inicial por bloco	50 STL
Intervalo alvo entre blocos	600 segundos (10 minutos)
Dificuldade inicial	4 (zeros hexadecimais iniciais no hash da prova)
Intervalo de halving	262.800 blocos
Número máximo de halvings	33
Maturação da recompensa de mineração	6 blocos
Limite de transações pendentes na mempool	1.000
Taxa protocolar (após ativação)	0,1% do valor transferido

Parâmetro	Valor
Teto da taxa	1 STL por transação
Unidade mínima	1 Saint = 0,00000001 STL

10.2 Recompensa e Halving

A recompensa de um bloco (`get_block_reward`) é calculada em Saints a partir da recompensa inicial, dividida por 2 elevado ao número de halvings já ocorridos ($((\text{índice_do_bloco} - 1) // \text{intervalo_de_halving})$), com truncamento inteiro. Ao atingir o 33º halving, a recompensa passa a ser zero — nesse ponto, considerando o intervalo alvo de 10 minutos por bloco, o encerramento da emissão por recompensa ocorreria após aproximadamente 262.800×33 blocos, o equivalente a cerca de 165 anos após o início da emissão, considerando um período de aproximadamente 5 anos por halving nesse ritmo alvo.

10.3 Maturação

Uma recompensa de mineração (transação `coinbase`) só se torna um UTXO disponível para gasto após a confirmação de 6 blocos subsequentes na cadeia principal (`process_matured_coinbases`), tanto no fluxo normal de adição de blocos quanto na reconstrução do conjunto de UTXOs ao trocar de cadeia (`rebuild_utxos`).

10.4 Taxas de Transação

A taxa de uma transação (`calculate_transaction_fee`) é zero enquanto a recompensa de mineração do bloco correspondente for maior que zero. Quando a recompensa de mineração chega a zero (após o 33º halving), a taxa passa a ser calculada como 0,1% do valor transferido, com um piso de 1 Saint e um teto de 1 STL.

10.5 Controle de Supply

A validade de uma transação `coinbase` (`valid_coinbase`) exige que seu valor seja exatamente igual à soma da recompensa esperada para o bloco com o total das taxas das demais transações do bloco, e que a emissão total acumulada, somada à nova recompensa, não ultrapasse o supply máximo definido.

11. Mempool

Transações válidas e ainda não incluídas em um bloco permanecem na mempool, mantida em memória (`Blockchain.current_transactions`) e persistida na tabela `mempool` do PostgreSQL. O tamanho da mempool é limitado a 1.000 transações pendentes (`mempool_full`); ao atingir esse limite, novas transações recebidas de outros nós são rejeitadas pela rota correspondente da API.

12. Comunicação entre Nós

Cada instância da aplicação identifica-se com um `node_id` gerado por `uuid4`. Nós remotos são registrados manualmente por meio de endereços HTTP fornecidos à API (`register_node`),

armazenados em memória durante a execução do processo — não há descoberta automática de outros nós.

A transmissão de transações e blocos (`broadcast_transaction`, `broadcast_block`) ocorre por meio de requisições HTTP POST diretas a cada nó conhecido, para as rotas `/transactions/receive` e `/blocks/receive`, respectivamente.

A resolução de conflitos entre cadeias concorrentes (`resolve_conflicts`) consulta a rota `/chain` de cada nó registrado e adota a primeira cadeia recebida que seja válida e que possua maior prova de trabalho acumulada do que a cadeia local, segundo o critério descrito na seção 13.

13. Critério de Escolha de Cadeia

Quando uma cadeia alternativa é avaliada (`choose_chain`), ela é aceita como nova cadeia principal se for estruturalmente válida (seção 6.5) e possuir um trabalho acumulado (`get_chain_work`, soma de 2 elevado à dificuldade de cada bloco) maior do que o da cadeia atual. Ao trocar de cadeia, o conjunto de UTXOs é reconstruído a partir da nova cadeia, e transações da cadeia anterior que não estejam presentes na nova cadeia e que continuem válidas retornam à mempool.

14. API

Todas as rotas abaixo estão implementadas em `api/app.py`. As rotas de interface web dependem de autenticação por sessão (`session['user_id']`), estabelecida via login.

14.1 Autenticação e Conta

Método	Rota	Finalidade
GET/POST	<code>/register</code>	Exibe formulário de cadastro (GET) ou cria um novo usuário e sua carteira, iniciando sessão (POST)
GET/POST	<code>/login</code>	Exibe formulário de login (GET) ou autentica um usuário existente por <code>login_id</code> e senha (POST)
GET	<code>/logout</code>	Encerra a sessão do usuário
GET	<code>/account</code>	Exibe dados da conta autenticada

14.2 Carteira e Transações (Interface Web)

Método	Rota	Finalidade
GET	<code>/home</code>	Exibe saldo e endereço da carteira do usuário autenticado
GET	<code>/wallet</code>	Exibe o endereço da carteira do usuário autenticado
GET	<code>/balance</code>	Exibe saldo e histórico recente de transações relacionadas ao endereço do usuário

Método	Rota	Finalidade
GET/POST	/send	Exibe formulário de envio (GET) ou cria, assina e envia à mempool uma transação para outro usuário cadastrado, identificado por seu login_id (POST)
GET	/receive	Exibe o identificador de login do usuário, utilizado para recebimento
GET	/transactions	Lista transações da cadeia relacionadas ao endereço do usuário
GET	/blocks	Exibe um bloco específico da cadeia, informado pelo parâmetro index, ou a página de consulta
GET/POST	/mine	Exibe a página de mineração (GET) ou executa a mineração de um novo bloco em nome do usuário autenticado (POST)

14.3 API de Rede e Dados (JSON)

Método	Rota	Finalidade
POST	/transactions/new	Cria uma nova transação a partir de entradas e saídas informadas, adiciona à mempool e transmite aos nós conhecidos
GET	/chain	Retorna a cadeia completa e seu comprimento
POST	/nodes/register	Registra uma lista de endereços de nós remotos
GET	/nodes/resolve	Executa o algoritmo de resolução de conflitos entre a cadeia local e as cadeias dos nós registrados
POST	/transactions/receive	Recebe uma transação de outro nó, valida-a, atualiza o conjunto de UTXOs disponíveis (removendo os UTXOs de entrada e criando os UTXOs de saída) e a inclui na mempool local
POST	/blocks/receive	Recebe um bloco de outro nó, valida-o e o incorpora à cadeia local
POST	/users	Cria um usuário e sua carteira via API, retornando os dados gerados
GET	/wallet/<user_id>/<login_id>	Retorna o endereço da carteira associada a um usuário
GET	/wallet/<user_id>/<login_id>/balance	Retorna o endereço e o saldo associado a um usuário

Método	Rota	Finalidade
GET	/transactions/<transaction_id>	Busca uma transação pelo identificador em toda a cadeia
GET	/blocks/<index>	Retorna um bloco específico pelo índice
POST	/transactions/send	Cria, assina e envia (à mempool e à rede) uma transação em nome do usuário autenticado, com destinatário e valor informados via JSON
GET	/debug/mempool	Retorna as transações atualmente pendentes na mempool

15. Persistência de Dados

O armazenamento é feito em PostgreSQL, com as seguintes tabelas geridas por `persistence/database.py`:

- **`blocks`**: índice, timestamp, transações (armazenadas como JSON), prova e hash do bloco anterior.
- **`utxos`**: identificador da transação, índice da saída, proprietário e valor — representando o conjunto de UTXOs disponíveis.
- **`wallets`**: endereço, chave pública (PEM) e chave privada criptografada (PEM cifrado com Fernet).
- **`mempool`**: identificador e conteúdo (JSON) das transações pendentes.
- **`users`**: identificador de usuário, identificador de login, nome de usuário, endereço da carteira e hash de senha.
- **`user\_sequence`**: controle do próximo login_id sequencial a ser atribuído.

16. Segurança

- **Senhas de usuário** são armazenadas como hash scrypt (parâmetros $n=16384$, $r=8$, $p=1$) com sal aleatório de 16 bytes, no formato `sal:hash`, ambos em hexadecimal.
- **Autenticação de sessão** é feita pelo mecanismo padrão de sessão do Flask, assinada com uma chave secreta definida na variável de ambiente `STELLA_SECRET_KEY`.
- **Assinatura de transações** utiliza ECDSA sobre a curva SECP256K1 com SHA-256, garantindo que apenas o titular da chave privada correspondente a um endereço possa autorizar o gasto dos UTXOs associados a ele.
- **Chaves privadas de carteira** são armazenadas de forma criptografada (Fernet) no banco de dados; a chave simétrica de criptografia é mantida em um arquivo local separado, fora do controle de versão.
- **Comunicação entre nós** ocorre por requisições HTTP diretas às rotas de recebimento de transações e blocos, sem mecanismos adicionais de autenticação entre nós além dos oferecidos pelo protocolo de transporte utilizado.

17. Fluxo de Operação Resumido

1. Um usuário se registra, recebendo uma carteira gerada automaticamente.
2. Ao enviar valor para outro usuário, uma transação é criada a partir dos UTXOs disponíveis do remetente, assinada com sua chave privada e enviada à mempool.
3. Um usuário autenticado pode minerar um novo bloco, que inclui as transações pendentes válidas e uma transação coinbase com a recompensa de mineração e as taxas acumuladas.
4. O bloco minerado é adicionado à cadeia local, persistido e transmitido aos demais nós conhecidos.
5. Nós que recebem transações ou blocos os validam antes de incorporá-los ao seu próprio estado.
6. Em caso de cadeias concorrentes entre nós, prevalece a cadeia com maior prova de trabalho acumulada.

18. Considerações Técnicas

A Stella implementa um protocolo de blockchain com modelo de UTXO, mineração por Proof of Work com dificuldade ajustável, emissão monetária limitada por halving sucessivo, e um mecanismo de taxas que se ativa automaticamente quando a recompensa de mineração se esgota. A camada de rede permite a propagação de transações e blocos entre nós registrados manualmente, com resolução de conflitos baseada no maior trabalho acumulado. As carteiras de usuário são geridas de forma custodial pelo próprio servidor da aplicação, que armazena as chaves privadas de forma criptografada.